

CLASE 4 · UNIDAD 1 · SEMANA 4

Word Embeddings

De token a vector

De dónde salen los vectores que representan texto, y por qué importan para RAG.

Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente



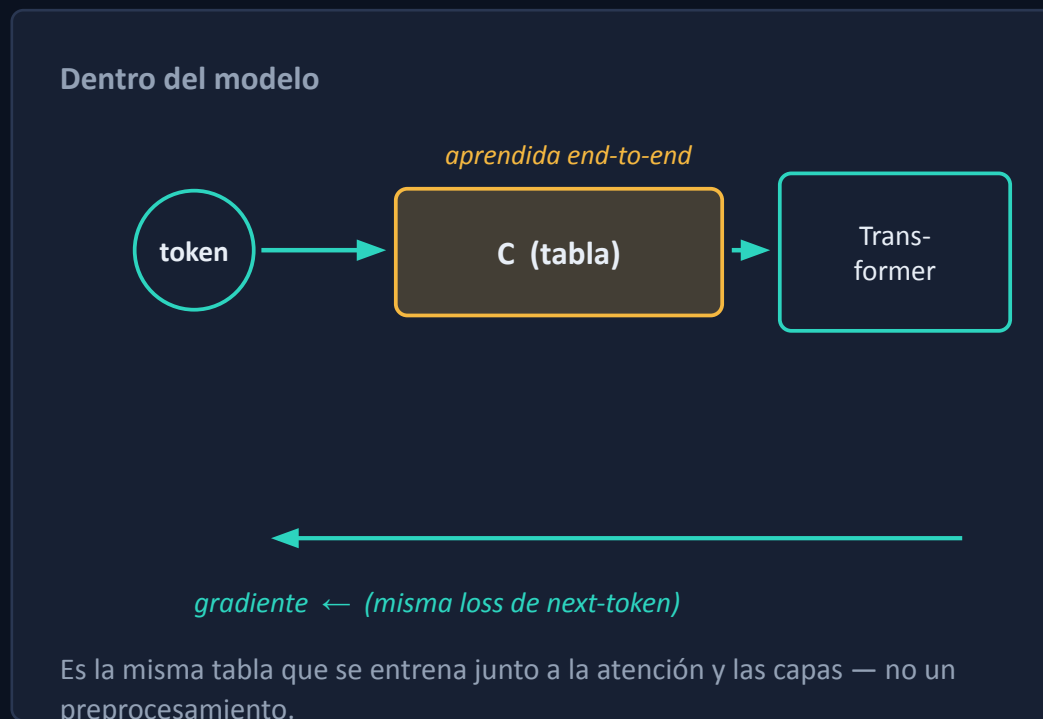
Los embeddings de un LLM se crean dentro del modelo

Dónde estamos. ya construyeron un Transformer. Su primera capa —la tabla de embeddings (nn.Embedding)— convierte cada token en un vector.

El punto a tener claro. en un LLM esa tabla se aprende dentro del modelo, junto con todo el resto. Arranca en valores random y se ajusta por backpropagation, con la misma loss de predecir el próximo token.

El modelo aprende su propia representación de cero, junto con todo lo demás.

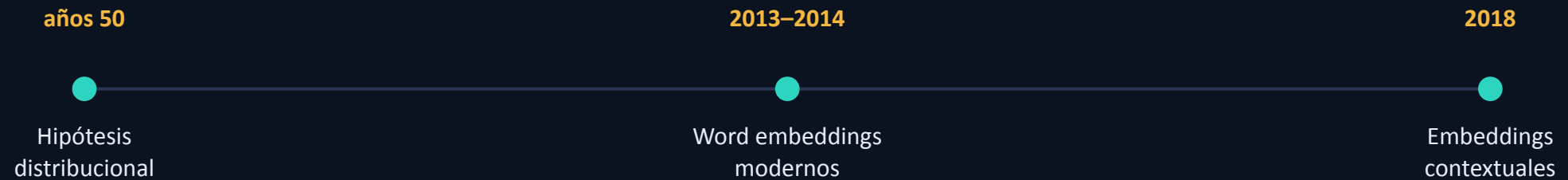
La clave. los embeddings del LLM no son un paso previo: son un parámetro más, entrenado end-to-end.



Los modelos que vemos en esta clase son para entender cómo funcionan los embeddings — no son el stack que vas a usar después. El objetivo es el mecanismo

Embeddings

Los embeddings no son un invento de los LLM. Vienen de mucho antes.



Idea: una palabra se conoce por su contexto (años 50). Los embeddings modernos (2013–14) son anteriores a los LLM; recién en 2018 se los aprende contextualmente, dentro de redes profundas grandes.

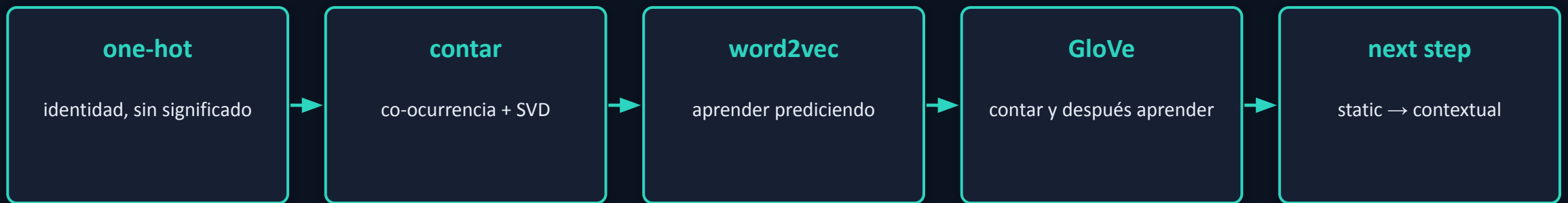
Estudiarlos solos sirve para dos cosas:

1. Entender por qué la capa de entrada de un LLM es una lookup table y qué representa.

2. El paradigma «modelo de embeddings standalone» sigue vivo — y es el que usa RAG.

Hoja de ruta - de texto a vectores

De dónde vienen los vectores. El arco completo, de a un paso.



Similitud coseno — la regla para medir «parecido» entre vectores — atraviesa todo el arco

Lo aplicable viene después. hoy vemos de dónde vienen los vectores. Más adelante, sentence embeddings — donde esto se vuelve directamente aplicable: el motor de recuperación de RAG.

Coseno: la regla para medir parecido

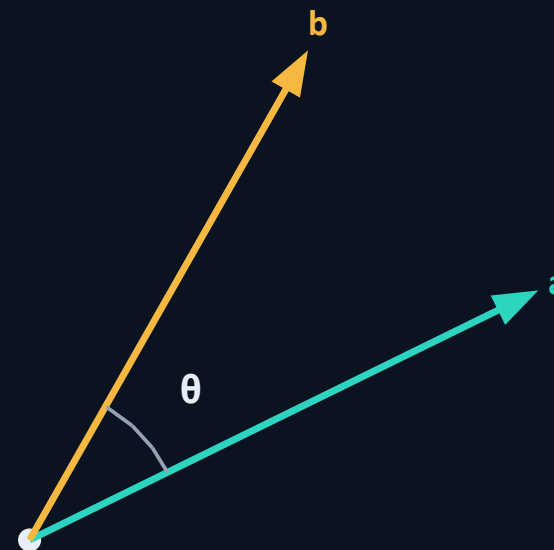
$$\cos(a, b) = (a \cdot b) / (\|a\| \|b\|)$$

El producto punto $a \cdot b$ crece cuando los vectores apuntan parecido; dividir por las normas cancela el largo de cada uno y deja pura dirección.

cos = 1 misma dirección · **cos = 0** ortogonales

Distancia coseno = $1 - \cos$. En búsqueda se piensa en «lo más cercano» (distancia baja = más cerca). Misma información, envase espejo.

Dirección, no tamaño. un vector que apunta igual que otro pero el doble de largo da $\cos = 1$ igual. Por eso comparamos orientación, no magnitud.



$$\cos(a, b) = \cos \theta$$

► **notebook** §1 — coseno con números y plot

De token a vector: el candidato ingenuo

La idea. con un vocabulario de tamaño V , cada token es un vector de V ceros con un único 1 en la posición de su ID. Es identidad: dice cuál token es, y nada más.

El problema, medido. entre dos one-hot el coseno siempre da 0: no comparten ninguna posición encendida. «gato» es tan parecido a «perro» como a «avión».

Y además: gigante y sparse (V puede ser 50.000, casi todo ceros).

El diagnóstico. one-hot codifica identidad y cero significado. Queremos lo contrario: vectores densos y de baja dimensión donde $\cos(\text{gato}, \text{perro})$ sea alto y $\cos(\text{gato}, \text{avión})$ bajo.

gato	1	0	0	0	0
perro	0	1	0	0	0
avión	0	0	1	0	0

$$\cos(\text{gato}, \text{perro}) = 0$$

$$\cos(\text{gato}, \text{avión}) = 0$$

todo ortogonal a todo

► **notebook** §2 — one-hot y su coseno

Un embedding es leer una fila: $\text{one_hot}(x) @ C$

La **mecánica**. un embedding es una tabla C de forma (V, d) : una fila por token, cada fila un vector de largo d . «Embeber» el token i es leer la fila i .

$$\text{one_hot}(i) @ C = C[i]$$

El 1 del one-hot «selecciona» la fila i y descarta el resto. El lookup no es más que un one-hot multiplicando la tabla — hecho de forma eficiente.

Un guiño. $\text{one_hot} @ C$ es, tal cual, la capa de entrada de un LLM — pero acá la estudiamos sola. Lo que sigue: ¿cómo la llenamos de significado? Contar o predecir.



► **notebook** §2 — `torch.allclose(C[i], one_hot @ C)`

¿Qué es «el significado» de una palabra?

El objetivo. para que un vector capture significado, primero hay que definir «significado» de una forma que sirva en la práctica.

La pista. ¿cómo hacemos nosotros, las personas, para saber qué palabras significan cosas parecidas?

Construir, no postular. no enunciamos la respuesta — la descubrimos con un ejemplo, y recién después le ponemos nombre.

El experimento

Vas a leer una palabra que (ojalá) no conocés, usada en varias oraciones. Prestá atención a cómo, sin diccionario, tu cabeza le encuentra el sentido.

Fijate cómo trabaja tu cerebro. →

¿Sabés qué significa «tezgüino»?

Mirá cómo se usa la palabra en distintos contextos:

Una botella de **tezgüino** está sobre la mesa.

A todos les gusta el **tezgüino**.

El **tezgüino** te emborracha.

Hacemos **tezgüino** a partir del maíz.



¿Podés entender qué significa?

Con el contexto, entendiste el significado

Una botella de tezgüino está sobre la mesa.

A todos les gusta el tezgüino.

El tezgüino te emborracha.

Hacemos tezgüino a partir del maíz.

tezgüino = una bebida alcohólica hecha de maíz.

¿Cómo lo hiciste?

Tu cerebro buscó otras palabras que encajan en los mismos contextos —por ejemplo, «vino»— y concluyó que tezgüino significa algo parecido a ellas.

El mecanismo. «significado parecido» lo dedujiste de «aparece en los mismos contextos». Esa equivalencia es toda la idea — vamos a hacerla explícita.

¿Qué otras palabras encajan? Armemos una tabla

Demos vuelta la pregunta: ¿qué otras palabras entran en estos mismos contextos?

	(1)	(2)	(3)	(4)	← contextos
(1) Una botella de ____ está sobre la mesa.					
	tezgüino	1	1	1	1
(2) A todos les gusta el ____.					
	ruidoso	0	0	0	0
(3) El ____ te emborracha.					
	aceite de motor	1	0	0	1
(4) Hacemos ____ a partir del maíz.					
	tortillas	0	1	0	1
	vino	1	1	1	0

1 si la palabra puede aparecer en ese contexto, 0 si no

La huella de contextos. cada fila anota, con 1 y 0, en qué contextos entra la palabra. Esa fila de unos y ceros es su primer «vector».

Filas parecidas → significados parecidos

La observación. tezgüino [1 1 1 1] y vino [1 1 1 0] tienen filas casi idénticas
→ significan cosas parecidas. Y de hecho: las dos son bebidas.

La hipótesis distribucional. palabras que aparecen en contextos parecidos tienen significados parecidos. La consecuencia: capturar significado = capturar contextos.

	(1)	(2)	(3)	(4)
tezgüino	1	1	1	1
ruidoso	0	0	0	0
aceite de motor	1	0	0	1
tortillas	0	1	0	1
vino	1	1	1	0

filas casi iguales → significados parecidos

Contar: primero, definir el contexto

El método más viejo. meter la información de contexto a mano, con estadística del corpus. Requiere dos decisiones:

1. ¿Qué es un «contexto»? las palabras dentro de una ventana de tamaño L alrededor de la central.

2. ¿Cómo medir la asociación? la más simple: contar cuántas veces aparecen juntas.

El mecanismo. deslizás la ventana por todo el corpus, sumando. Recién eso arma la tabla — la vemos en la próxima.



contexto de «gato»: vi, un, gris, jugando

+1 en la fila de «gato» por cada una

La matriz de co-ocurrencia: conteo crudo

El resultado. repetí el conteo por todo el corpus → una tabla palabra × contexto. La versión más simple: $M[i, j]$ = cuántas veces co-ocurren.

El problema del conteo crudo. lo domina la frecuencia. Palabras muy comunes («de», «la», «que») co-ocurren con todo, así que sus conteos son enormes aunque no aporten significado.

Antes de comprimir. cada fila ya es un vector (la tabla de tezguino a escala). Pero primero conviene arreglar el efecto de la frecuencia — con PPMI.

	oil	crude	wheat	corn
oil	0	42	3	2
crude	42	0	4	3
wheat	3	4	0	55
corn	2	3	55	0

$M[i, j]$ = veces que j cae en la ventana de i

► **notebook** §3 — co-ocurrencia desde cero

La mejora: PPMI

Positive Pointwise Mutual Information. en vez de «cuántas veces co-ocurren», medir cuánto MÁS co-ocurren de lo esperable si las palabras fueran independientes.

$$\text{PMI}(w, c) = \log \left(\frac{P(w, c)}{P(w) \cdot P(c)} \right)$$

$$\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c))$$

observado $P(w, c)$ **esperado** $P(w) \cdot P(c)$

El recorte a 0. los PMI negativos vienen de pares que casi nunca aparecen juntos — ruido poco confiable. Se descartan.

conteo crudo vs PPMI

par

conteo

PPMI

«de» + gato

alto

≈ 0

co-ocurre por frecuencia, no por significado

«crudo» + petróleo

medio

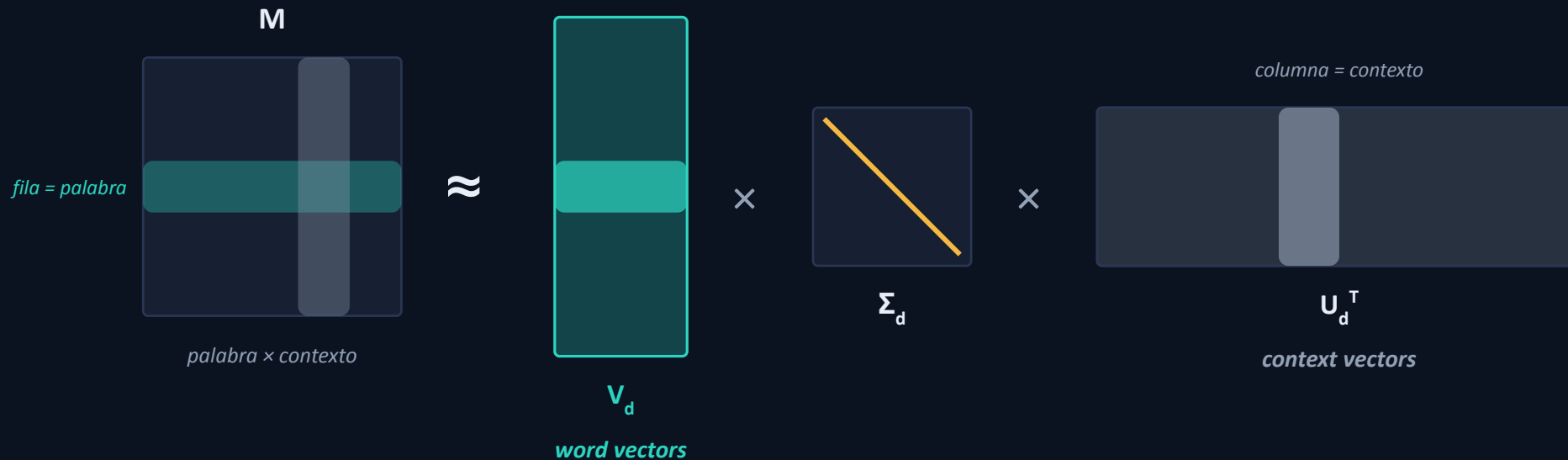
alto

asociación genuina

Qué logra. saca el efecto de la frecuencia y deja la asociación genuina. Es el estándar pre-neuronal para similitud distribucional.

Comprimir con SVD

Sobre esa matriz —cruda o, mejor, PPMI— aplicás SVD para bajar la dimensión.



SVD es ortogonal al conteo. comprime la matriz que le des; con PPMI, los vectores salen mejores.

El pago. cada palabra pasa de $|\text{contextos}|$ dimensiones a d . Los word vectors salen de V_d .

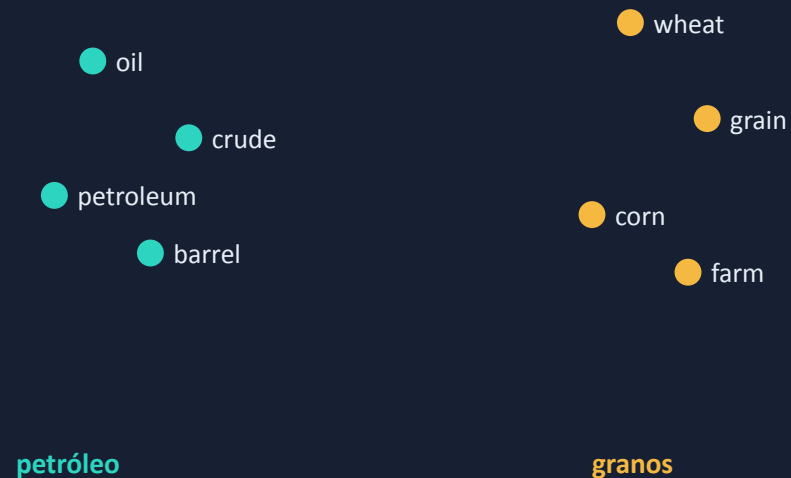
Qué sale del conteo, y su techo

El resultado. co-ocurrencia + PPMI + SVD te da un vector por palabra. Proyectado a 2D, lo parecido se agrupa: petróleo por un lado, granos por otro.

Sin decirle a nadie qué significan, el puro conteo de contextos las agrupa por tema.

El techo. emerge geometría, pero es ruidosa, 2D aplasta el resto, y sobre todo no escala: SVD sobre millones de palabras no va. → aprender en vez de contar.

co-ocurrencia + PPMI + SVD → 2D



► **notebook** §3 — SVD, PPMI y el scatter

El truco central: una tarea descartable

La jugada. no modelamos el significado de frente. Inventamos una tarea de predicción descartable — y los vectores que el modelo necesita para resolverla son nuestros embeddings.

Self-supervised. las «etiquetas» salen gratis del propio texto: no hay que anotar nada a mano.

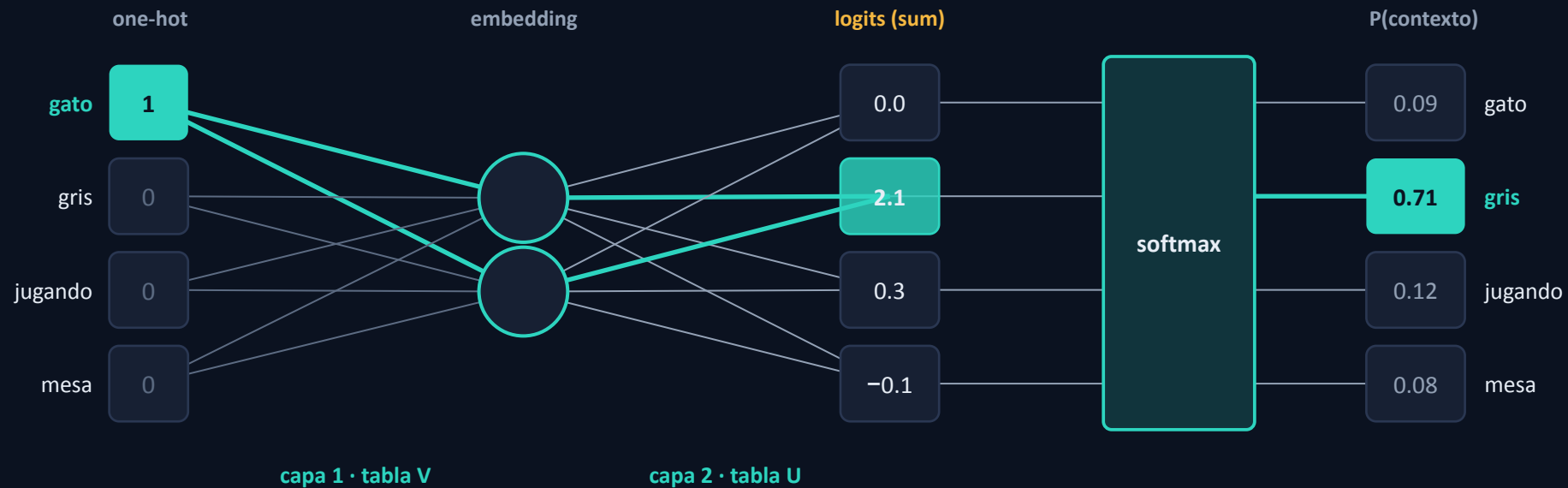
La frase para retener. la tarea es la excusa; los vectores son el premio.
Word2vec (Mikolov et al., 2013) es el salto de contar a aprender.



la tarea se tira; los vectores quedan

La red por dentro: de one-hot a probabilidad

one-hot → tabla V (embedding) → tabla U → logits (sum) → softmax → P(contexto). Sin dibujar los pesos.



El final: logits → softmax → probabilidad. la tabla U puntúa el embedding contra cada palabra (los logits); el softmax los vuelve una distribución. Cada palabra tiene una fila en V (central) y otra en U (contexto), y su producto punto es el score del par — esos pesos son los embeddings.

Skip-gram: predecir el contexto

La tarea concreta. dada una palabra central, predecir sus vecinas dentro de una ventana. Deslizando la ventana por todo el corpus se generan millones de pares (central, contexto).

CBOW, la variante espejo: desde el contexto, predecir la central. Skip-gram es la que se suele enseñar.

Sin anotar nada. los pares salen del texto solo: cada palabra y sus vecinas. Es la materia prima con la que el modelo va a aprender los vectores.



► **notebook** §4 — entrenar word2vec (gensim, text8)

Cómo se entrena: lo que ya saben, aplicado acá



$$\text{score}(\text{centro}, \text{contexto}) = \mathbf{v}_{\text{centro}} \cdot \mathbf{u}_{\text{contexto}}$$

Para que el producto punto sea alto entre palabras que co-ocurren, el modelo se ve forzado a darles vectores parecidos a las que viven en contextos parecidos — y el significado cae de la geometría.

Lo nuevo no es la maquinaria. es sobre qué corre: el softmax es sobre el vocabulario, la likelihood es la de los contextos reales, y lo que se aprende son los vectores.

El cuello de botella: el softmax mira todo el vocabulario

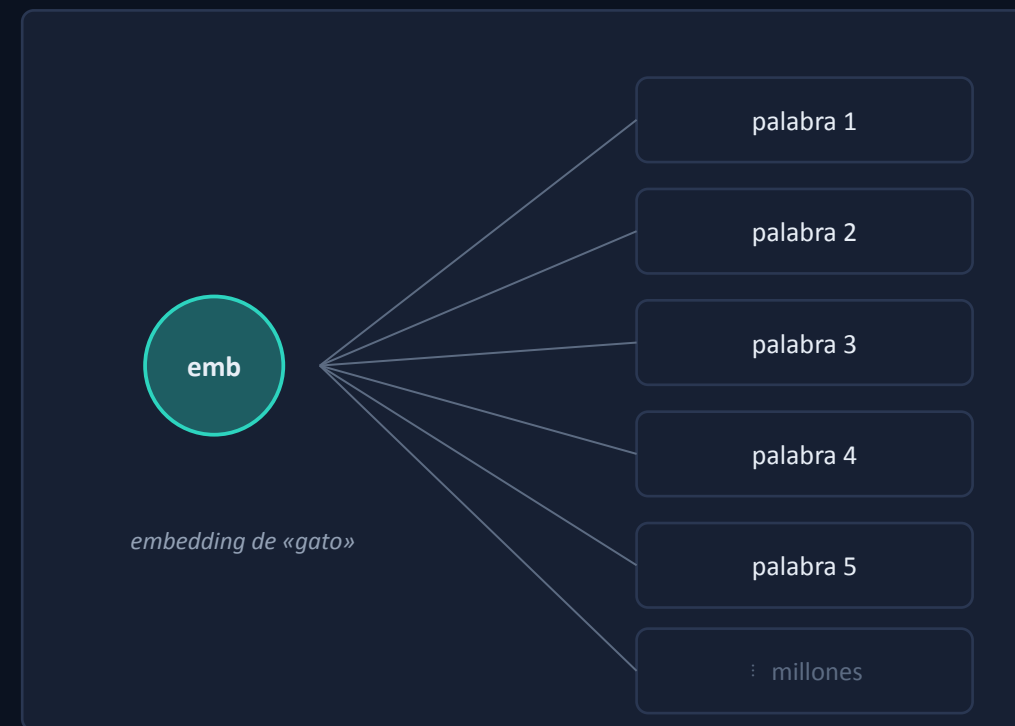
El problema. el softmax de la salida normaliza sobre las $|V|$ palabras del vocabulario — y lo hace en CADA par (central, contexto). Con $|V|$ de millones, es carísimo.

La cuenta (un ejemplo real):

$|V|$ 3.000.000 × d 100 × 2 tablas

= 600.000.000 pesos

El costo. normalizar el softmax sobre todo el vocabulario en cada par es $O(|V|)$ — proporcional a los millones de palabras. Impracticable a escala.



puntuar contra las $|V|$ palabras → softmax

La idea: de «elegir entre V» a «real o falso»

ANTES · SOFTMAX

¿cuál de las $|V|$ palabras es el contexto?

Una elección entre millones de opciones: hay que puntuar y normalizar sobre TODO el vocabulario.

AHORA · NEGATIVE SAMPLING

¿este par (central, contexto) es real?

Una pregunta sí/no: se responde con la sigmoide, sin mirar el resto del vocabulario.

Cómo se arma. el par real (central + su contexto de verdad) es un ejemplo positivo (etiqueta 1). Después elegís k palabras al azar como contextos falsos: ejemplos negativos (etiqueta 0).

El cambio de marca. de un problema de V opciones (softmax) a $k+1$ problemas binarios (sigmoide). Cada uno sólo pregunta «¿co-ocurren?» — y eso no necesita mirar todo el vocabulario.

Negative sampling: el positivo + k negativos

En cada paso. en vez de tocar las $|V|$ salidas, sólo mirás el contexto real (positivo) y k palabras random (negativos). El resto del vocabulario no se toca.

El objetivo. subir el score (producto punto $\rightarrow$ sigmoide) del par real, y bajar el de los k falsos.

El ahorro. de actualizar millones de pesos por paso a tocar sólo k+1 palabras. La misma señal —co-ocurre / no co-ocurre— con una fracción del cómputo.

central: «gato»

gris

1 real

mesa

0 falso

avión

— no se toca

sol

0 falso

jugando

— no se toca

... resto del vocab

— no se toca

sólo k+1 palabras por paso

► **notebook** §7 — skip-gram + negative sampling desde cero

No es un softmax más chico

El punto fino. no cambiamos «V opciones» por «k+1 opciones» en un softmax. Son k+1 preguntas sí/no independientes — y eso cambia el tipo de problema.

SOFTMAX · elegí uno entre todos

¿cuál de las V palabras es el contexto?

gris		0.55
mesa		0.20
sol		0.15
avión		0.10

$\Sigma = 1.00$ compiten: si una sube, otras bajan

NEGATIVE SAMPLING · ¿real o falso? (por separado)

¿este par (gato, ·) co-ocurre?

(gato, gris)	$\sigma = 0.98$	→ 1 (sí)
(gato, mesa)	$\sigma = 0.03$	→ 0 (no)
(gato, sol)	$\sigma = 0.05$	→ 0 (no)

sin Σ independientes: no suman 1, cada par se juzga solo

Por qué esto ahorra. el costo del softmax es el denominador: sumar sobre TODO el vocabulario para normalizar. La sigmoide no normaliza contra nada — sólo mira ese par. Por eso alcanza con k+1, y el problema pasa de multiclase a binario.

El resultado: $O(k)$ en vez de $O(V)$

SOFTMAX COMPLETO

$O(|V|)$

~600.000.000 pesos por paso

NEGATIVE SAMPLING

$O(k)$

~300 pesos por paso

El detalle que lo hizo posible. misma geometría, misma señal de co-ocurrencia — pero entrenable sobre miles de millones de tokens. Sin este truco, word2vec a escala no existiría.

k típico: 5 a 20 negativos por positivo.

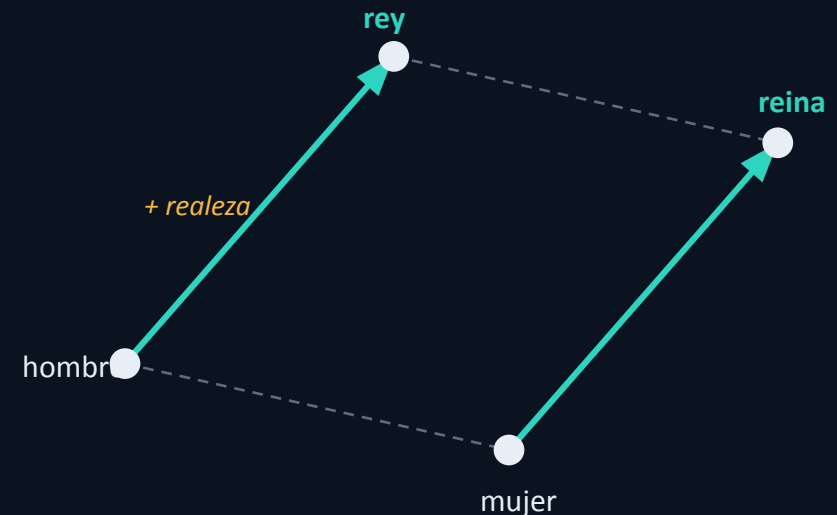
El resultado famoso: analogías como aritmética

La estructura lineal. las direcciones del espacio codifican relaciones.
Restarle a «rey» la dirección de «hombre» y sumarle la de «mujer» cae cerca de «reina».

$\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$

Lo mismo con capital-de-país o singular-plural. No era obvio que una tarea de juguete diera analogías algebraicas.

La prueba. el texto puede vivir en un espacio donde significado = geometría, y las relaciones son direcciones.

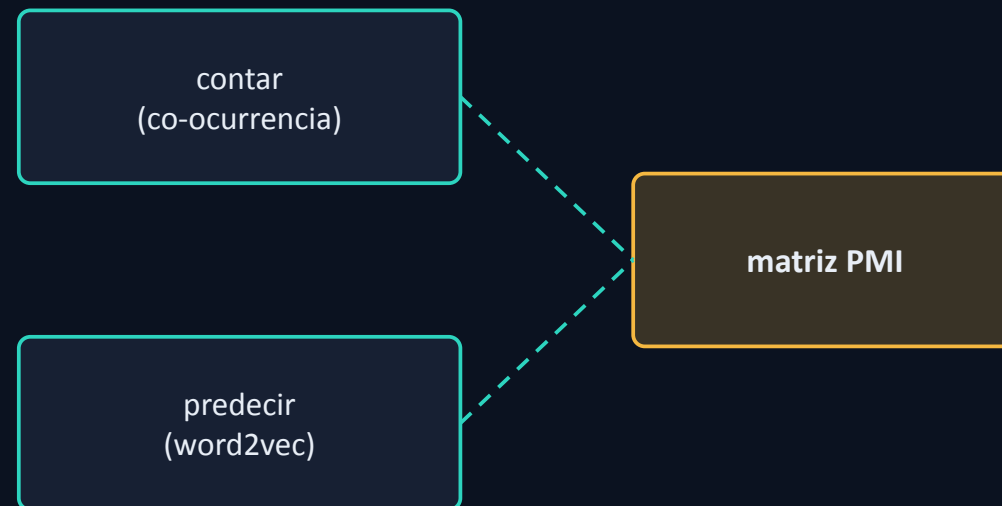


Predecir es, en secreto, contar

El descubrimiento. skip-gram con negative sampling no es tan distinto de contar: Levy & Goldberg (2014) mostraron que factoriza implícitamente una matriz de PMI (la misma idea de contar que vimos recién).

Los dos caminos que separamos —contar y predecir— resultan ser dos caras de lo mismo.

El puente a GloVe. si predecir es en el fondo contar, ¿se puede diseñar un método que use los conteos de frente y aún así dé la geometría lineal? Sí: GloVe.



las dos llegan a la misma geometría

Dos familias: contar vs predecir

Antes de GloVe hay dos formas de armar embeddings — cada una con su fortaleza y su debilidad.

CONTAR · Parte 2

- + entrenamiento rápido
- + usa toda la estadística global del corpus
- da peso desproporcionado a los conteos grandes

→ es justo lo que PPMI vino a mitigar

PREDECIR · word2vec

- + escala con el tamaño del corpus
- + captura patrones más ricos
- usa la estadística de forma ineficiente

→ 500 co-ocurrencias = 500 eventos separados, no el número 500

GloVe: lo mejor de las dos. contar (global, eficiente) + aprender (patrones ricos) — arreglando las dos debilidades. Por eso «contar y después aprender».

Del conteo al cociente

1. El conteo (Parte 2):

X_{ik} = veces que k cae en la ventana de i

2. Probabilidad de co-ocurrencia:

$$P(k | i) = X_{ik} / X_i$$

de todas las veces que aparece i , qué fracción tiene a k cerca.

3. La jugada de GloVe:

no mires $P(k | i)$ sola — mirá el cociente entre dos palabras.

El objeto de GloVe. ese cociente es lo que va a codificar en los vectores.
En la próxima lo calculamos para palabras concretas.

EL COCIENTE

$$P(k | \text{ice}) / P(k | \text{steam})$$

$\gg 1$ *k va más con ice*

$\ll 1$ *k va más con steam*

≈ 1 *no distingue*

comparar dos probabilidades, no mirar una sola

El cociente en acción: ice vs steam

Calculemos ese cociente para varias palabras *k* de prueba — ¿qué tan asociada está cada *k* con ice frente a steam?

k	P(k ice)	P(k steam)	cociente	
solid	alto	bajo	8.9	va con ice
gas	bajo	alto	0.085	va con steam
water	alto	alto	≈ 1	con las dos
fashion	bajo	bajo	≈ 1	con ninguna

Pennington, Socher & Manning, 2014 — Stanford.

Lo que el cociente ve (y lo que no). da ≈1 cuando las dos probabilidades se parecen — sea alto/alto (con las dos) o bajo/bajo (con ninguna): la magnitud queda afuera. Lo que sí marca es la dirección (solid con ice, gas con steam). GloVe ajusta los vectores para reproducir ese cociente.

El diseño al revés

La **idea rara de GloVe**. no parte de una tarea (como word2vec) — parte de la geometría que quiere tener, y la convierte en la especificación.

Lo que hace útiles a los vectores. las diferencias son direcciones que codifican relaciones: rey – hombre es una dirección.

Así que eso ES el pedido. la diferencia de los dos vectores tiene que ser el eje que distingue i de j, y proyectar una palabra sobre él debe recuperar el cociente.

El giro. las analogías no son una sorpresa emergente — son la restricción de diseño.

EL PEDIDO, ESCRITO

$$(v_i - v_j) \cdot u_k = \log(P(k|i) / P(k|j))$$

resta a la izquierda, **división** a la derecha.

Una función las conecta: la exponencial. Pedir estructura lineal FUERZA un modelo log-bilineal.

Un detalle lindo

Por eso «desaparece» la tercera palabra: una vez forzada la forma exponencial, satisfacer cada celda por separado — $v_i \cdot u_j \approx \log X_{ij}$ — satisface TODOS los cocientes a la vez. La tercera palabra especificó el requisito; ajustar celda por celda lo cumple.

El objetivo: del cociente al producto punto

$$\mathbf{v}_i \cdot \mathbf{u}_k + b_i + b_k = \log X_{ik}$$

$\mathbf{v}_i$ = fila de V (central) $\mathbf{u}_k$ = fila de U (contexto)

El modelo. el producto punto de dos vectores, más dos bias, iguala el log del conteo.

Los bias. absorben la frecuencia de cada palabra — la normalización que antes hacía PPMI, ahora aprendida.

La función de peso $f(\mathbf{x})$. baja los pares raros (ruido), capea los frecuentes («the»), y $f(0)=0 \rightarrow$ saltea los ceros: el costo escala con los no-ceros, no con $\mathbf{v}^2$.

El vector final. cada palabra suma su vector de V y su de U — un pequeño ensamble.

Regresión sobre la matriz

Mínimos cuadrados ponderados: el producto punto predicho contra el log del conteo observado. Una pasada al corpus arma la matriz; después se itera sólo sobre los no-ceros.

```
glove = api.load(  
    "glove-wiki-gigaword-100")  
  
glove.most_similar("king")
```

► **notebook** §5 — GloVe pre-entrenado, coseno, analogías

GloVe no es una red

Nada de red. sin capa oculta, sin no-linealidad, sin softmax. Es un modelo bilineal — una regresión (mínimos cuadrados) sobre la matriz ya contada, entrenada por SGD. Profundidad cero.

El contraste. word2vec era una red que se entrena sobre el stream de tokens; GloVe es una regresión sobre una matriz. Mismo layout de parámetros, señal de entrenamiento distinta.

La unificación. los tres —SVD sobre PPML, skip-gram, GloVe— factorizan la misma matriz de asociación corregida por frecuencia. Difieren en cómo, no en qué.

word2vec	GloVe
red	modelo bilineal
softmax sobre $ V $	producto punto $\rightarrow$ escalar
stream de tokens	matriz precomputada
capas de atención	profundidad cero

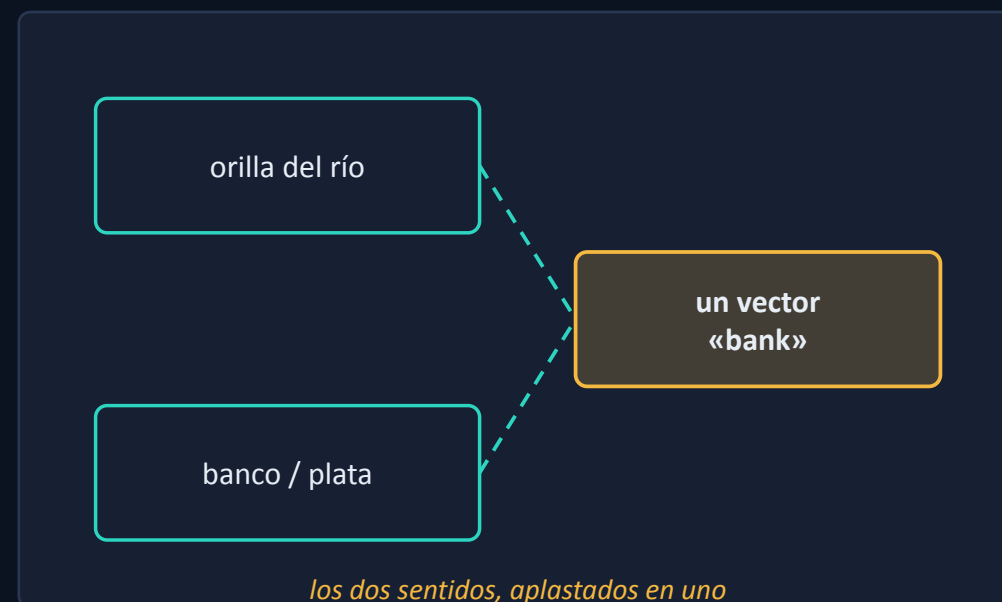
Epílogo honesto. con hiperparámetros parejos, la ventaja empírica de GloVe se diluye (Levy, Goldberg & Dagan, 2015). Lo que queda es conceptual: por qué debería existir la estructura lineal.

El techo compartido: static embeddings

Todos comparten un límite. one-hot, contar, word2vec y GloVe dan static embeddings: un vector fijo por palabra, ciego al contexto.

El problema: polisemia. «bank» tiene dos sentidos (río / finanzas en ingles) pero un solo vector. No puede sostener los dos a la vez: los promedia o deja ganar al más común.

El techo. un único vector no alcanza para una palabra con varios sentidos. Para romperlo hace falta que el vector dependa del contexto.



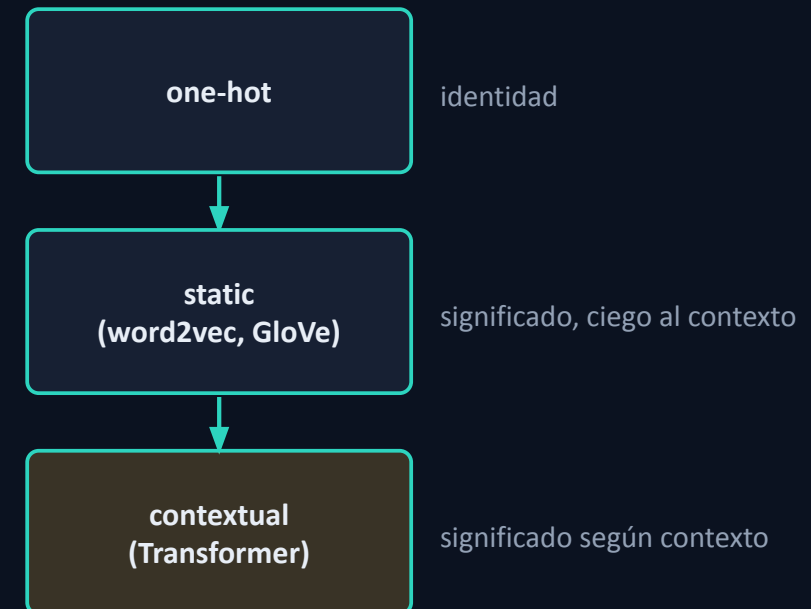
► **notebook** §5 — vecinos de «bank»

La resolución: embeddings contextuales

El siguiente escalón. un vector distinto por token según sus vecinos.
«bank» en «río» y en «plata» reciben vectores distintos.

Y eso ya lo construyeron. un vector que depende del contexto es exactamente lo que hace self-attention — el Transformer que ya construyeron.

El arco entero. one-hot (identidad) → static (significado, ciego al contexto) → contextual (significado que depende del contexto).

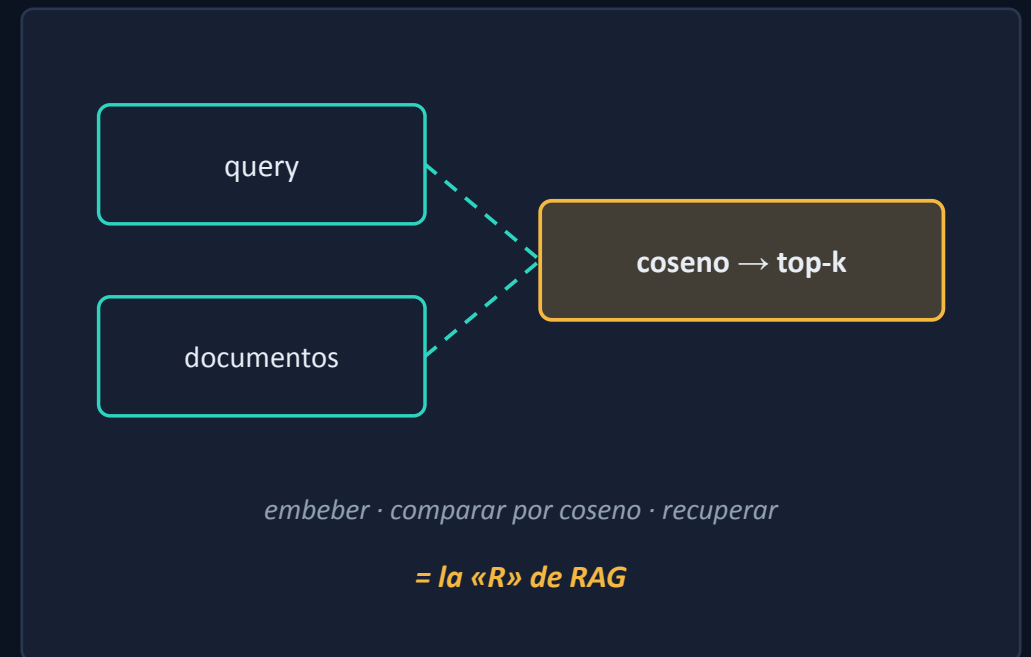


El paréntesis, cerrado: esto es RAG

No es sólo historia. word2vec y GloVe son embedding models standalone: un modelo entrenado por su cuenta para producir vectores. Ese paradigma está vivo — y es el motor de recuperación de RAG.

La operación es la misma. `most_similar` —buscar los vectores más cercanos por coseno— es, literalmente, la recuperación en RAG.

Lo que sigue. en sentence embeddings pasamos de la palabra a la oración, y montamos búsqueda semántica sobre eso. Es la «R» de RAG (Semana 8).



Síntesis: cómo llenar la tabla con significado

El mapa del principio, ahora recorrido entero.

enfoque	idea	qué da
one-hot	identidad	vectores ortogonales — cero significado
contar (co-oc. + SVD)	hipótesis distribucional	geometría, pero ruidosa y no escala
word2vec	aprender resolviendo una tarea	analogías, self-supervised
GloVe	contar y después aprender	lo mejor de los dos

El arco. one-hot (identidad) → static (significado, ciego al contexto) → contextual (Transformer). Todos static salvo el último.

Trabajo y lecturas

NOTEBOOK — N1 · WORD EMBEDDINGS

El arco de hoy, ejecutable de punta a punta.

- 1 Coseno con números y plot (§1)
- 2 one-hot → lookup: one_hot @ C (§2)
- 3 Co-ocurrencia + SVD + PPMI desde cero (§3)
- 4 word2vec real con gensim, text8 (§4)
- 5 GloVe: coseno, most_similar, analogías (§5)
- 6 skip-gram + negative sampling desde cero (§7)

LECTURAS

Núcleo

- Lena Voita — «Word Embeddings» (NLP Course For You)
- CS224N (Stanford) — Lectures 1 y 2 (Manning)
- Jay Alammar — The Illustrated Word2vec

Para afianzar

- StatQuest — «Word Embedding and Word2Vec, Clearly Explained» (video)

Lo que sigue. sentence embeddings y búsqueda semántica — la «R» de RAG.